

SCD, Performance e Views Analíticas no PostgreSQL

BUSINESS INTELLIGENCE

POSTGRESQL

NÍVEL INTERMEDIÁRIO

UNICEUB

Bem-vindo a esta aula prática! Neste exercício guiado, você irá trabalhar com o cenário real do **Supermercado ABC** para aprender na prática os conceitos fundamentais de Business Intelligence aplicados ao PostgreSQL.

Em aproximadamente 2 horas, você passará por todas as etapas: desde a importação de um dataset com 1 milhão de registros até a medição concreta de ganhos de performance com índices.

Professor: José Antonio de Paiva Júnior | **Duração estimada:** 2 horas | **Nível:** Intermediário / Didático



Objetivos da Aula Prática

Ao final desta atividade, você terá vivenciado cada etapa do ciclo de desenvolvimento de um ambiente analítico real. Os objetivos foram planejados para progredir de forma lógica: primeiro estruturar os dados, depois modelar o histórico, criar camadas de análise e, por fim, otimizar a performance. Cada parte reforça conceitos que serão utilizados nas partes seguintes.

1

Importar Dataset

Carregar 1 milhão de registros de vendas no PostgreSQL a partir de um arquivo CSV.

2

Implementar SCD Tipo 2

Criar um histórico rastreável de mudanças em dimensões, preservando versões anteriores dos dados.

3

Criar Views Analíticas

Desenvolver Views e Materialized Views para simplificar consultas e pré-processar agregações.

4

Testar Performance

Medir e comparar o tempo de execução de consultas antes e depois da criação de índices.

Cenário do Exercício: Supermercado ABC

A empresa **Supermercado ABC** enfrenta desafios clássicos de um ambiente analítico em crescimento. Com milhões de transações acumuladas ao longo dos anos, as consultas utilizadas nos dashboards do Power BI começaram a apresentar lentidão inaceitável para a equipe de gestão. Além disso, mudanças nas categorias de produtos ao longo do tempo precisam ser rastreadas para garantir a integridade histórica das análises.

Como analista de dados responsável, você assumirá três frentes de trabalho simultâneas: **melhorar a modelagem dimensional** com o uso de SCD Tipo 2, **criar um histórico confiável** das alterações de produtos, e **otimizar as consultas** por meio de Views e índices estratégicos. Este é exatamente o tipo de trabalho realizado por profissionais de BI em ambientes de produção.

Problema 1

Milhões de vendas acumuladas gerando consultas lentas nos dashboards.

Problema 2

Alterações de categorias de produtos sem histórico rastreável.

Solução

SCD Tipo 2 + Views otimizadas + índices estratégicos.



Importação do Dataset (15 min)

O primeiro passo é criar a estrutura da tabela principal de fatos e importar o arquivo `supermercado_abc_1M.csv`. A tabela `fato_vendas` representa o coração do modelo dimensional: ela armazena cada transação de venda com todas as dimensões relevantes desnormalizadas para facilitar as consultas analíticas.

Observe com atenção os tipos de dados escolhidos — o uso de `NUMERIC(10,2)` para valores monetários é essencial para evitar problemas de arredondamento comuns com tipos `FLOAT`.

```
CREATE TABLE fato_vendas (  
  id_venda      BIGINT,  
  data_venda    DATE,  
  id_produto    INTEGER,  
  produto       VARCHAR(100),  
  categoria     VARCHAR(50),  
  quantidade    INTEGER,  
  valor_unitario NUMERIC(10,2),  
  valor_total   NUMERIC(10,2),  
  cidade        VARCHAR(50)  
);
```



Validação dos Registros (5 min)

Antes de prosseguir com qualquer análise, é fundamental validar que todos os dados foram importados corretamente. Esta é uma prática essencial em projetos de BI: nunca assuma que a importação foi bem-sucedida sem verificação. Um número incorreto de registros pode invalidar todas as análises subsequentes e gerar conclusões equivocadas para a gestão.

```
SELECT COUNT(*) FROM fato_vendas;
```

1M

Registros Esperados

Resultado que confirma a importação completa e sem falhas do dataset.

9

Colunas na Tabela

Atributos que compõem cada registro de venda no modelo desnormalizado.



Atenção: Se o resultado for diferente de 1.000.000, verifique se houve linhas rejeitadas durante a importação. Execute também `SELECT * FROM fato_vendas LIMIT 5;` para confirmar que os dados estão legíveis e nos formatos corretos.



Slowly Changing Dimensions — O Conceito

O conceito de **Slowly Changing Dimension (SCD) Tipo 2** é um dos pilares do Data Warehousing. Em vez de simplesmente sobrescrever um registro quando há uma mudança (como faria um sistema transacional), o SCD Tipo 2 **preserva o histórico** criando uma nova linha para cada versão do registro. Desta forma, é possível saber não apenas o estado atual de uma dimensão, mas também como ela era em qualquer ponto do passado.

No nosso cenário, o produto **"Refrigerante Cola 2L"** mudou de categoria: antes pertencia a *Refrigerantes* e passou a pertencer a *Bebidas* a partir de janeiro de 2025. Sem SCD Tipo 2, seria impossível responder perguntas como: "Qual foi o faturamento da categoria Refrigerantes em 2024?" — pois essa informação seria perdida após a atualização.

1

Versão Anterior

Refrigerante Cola 2L na categoria **Refrigerantes** de 01/01/2024 até 01/01/2025
ativo = FALSE

2

Mudança Detectada

Produto muda de categoria em **02/01/2025**. O registro antigo é encerrado e um novo é criado.

3

Nova Versão

Refrigerante Cola 2L na categoria **Bebidas** de 02/01/2025 em diante
ativo = TRUE



Implementando a Dimensão Produto

Agora vamos colocar a teoria em prática. A tabela `dim_produto` é o coração do SCD Tipo 2. (`id_dim_produto` SERIAL) separada da chave de negócio (`id_produto` INTEGER). Isso é fundamental: o mesmo `id_produto` pode ter múltiplas linhas, cada uma representando uma versão diferente do produto ao longo do tempo. Os campos `data_inicio`, `data_fim` e `ativo` controlam a validade de cada versão.

```
CREATE TABLE dim_produto (  
  id_dim_produto SERIAL PRIMARY KEY,  
  id_produto     INTEGER,  
  nome_produto   VARCHAR(100),  
  categoria      VARCHAR(50),  
  data_inicio    DATE,  
  data_fim       DATE,  
  ativo          BOOLEAN  
);
```

Com a estrutura criada, insira as duas versões do produto. A primeira representa o histórico (categoria antiga), e a segunda representa a versão atual ativa. Note que o campo `data_fim` da versão atual é `NULL` — isso indica que o registro está vigente e ainda não foi encerrado.

Inserir Versão Original

```
INSERT INTO dim_produto  
(id_produto, nome_produto,  
 categoria, data_inicio,  
 data_fim, ativo)  
VALUES  
(10, 'Refrigerante Cola 2L',  
 'Refrigerantes',  
 '2024-01-01',  
 '2025-01-01', FALSE);
```

Inserir Nova Versão

```
INSERT INTO dim_produto  
(id_produto, nome_produto,  
 categoria, data_inicio,  
 data_fim, ativo)  
VALUES  
(10, 'Refrigerante Cola 2L',  
 'Bebidas',  
 '2025-01-02',  
 NULL, TRUE);
```

Para validar o histórico criado, execute a consulta abaixo e confirme que aparecem **duas linhas** para o mesmo `id_produto = 10`, cada uma com sua versão de categoria e período de vigência.

```
SELECT * FROM dim_produto WHERE id_produto = 10;
```



Criando Views Analíticas

Uma **View** no PostgreSQL é uma consulta armazenada que pode ser tratada como se fosse uma tabela. Ela não armazena dados fisicamente — toda vez que você consulta uma View, o PostgreSQL executa a consulta original por baixo dos panos. O grande benefício é a **abstração**: os analistas e ferramentas de BI podem consultar a View sem precisar conhecer a lógica complexa por trás dela. Isso promove reuso de código, padronização das métricas e facilita a manutenção.

```
CREATE VIEW vw_vendas_categoria AS
SELECT
  categoria,
  SUM(valor_total) AS faturamento,
  SUM(quantidade) AS total_itens
FROM fato_vendas
GROUP BY categoria;
```

Após criar a View, você pode consultá-la exatamente como consultaria uma tabela comum. Ordene pelo faturamento decrescente para identificar rapidamente as categorias mais lucrativas do Supermercado ABC:

```
SELECT * FROM vw_vendas_categoria
ORDER BY faturamento DESC;
```



Abstração de Complexidade

O analista consulta `vw_vendas_categoria` sem precisar escrever JOINS ou agregações manualmente a cada vez.



Dados Sempre Atualizados

A View sempre retorna os dados mais recentes da tabela base, pois a consulta é reexecutada a cada acesso.



Controle de Acesso

É possível conceder acesso à View sem expor a tabela original, protegendo colunas sensíveis.

Materialized Views — Performance Pré-computada

Enquanto uma View comum reexecuta a consulta a cada acesso, uma **Materialized View** armazena o resultado fisicamente no disco. Isso significa que consultas sobre grandes volumes de dados — como agregações mensais de 1 milhão de registros — são executadas muito mais rapidamente, pois o PostgreSQL lê dados já processados em vez de recalculá-los tudo do zero. A contrapartida é que os dados podem ficar desatualizados e precisam ser atualizados manualmente com o comando `REFRESH`.

```
CREATE MATERIALIZED VIEW mv_vendas_mensais AS
SELECT
  DATE_TRUNC('month', data_venda) AS mes,
  SUM(valor_total) AS faturamento
FROM fato_vendas
GROUP BY mes;
```

Para consultar e posteriormente atualizar a Materialized View:

Consultar

```
SELECT *
FROM mv_vendas_mensais
ORDER BY mes;
```

Atualizar Dados

```
REFRESH MATERIALIZED VIEW
mv_vendas_mensais;
```



 **Quando usar Materialized Views?** Utilize-as em consultas pesadas que são executadas com frequência, mas cujos dados subjacentes mudam com pouca frequência — por exemplo, relatórios diários ou semanais. Elas são ideais para alimentar dashboards do Power BI com dados pré-agregados, reduzindo drasticamente o tempo de carregamento dos relatórios.



Teste de Performance com EXPLAIN ANALYZE

Esta é a parte mais reveladora do exercício. O comando `EXPLAIN ANALYZE` é uma ferramenta poderosa do PostgreSQL que mostra **como** o banco de dados executa uma consulta — quais estratégias ele usa, quais tabelas acessa e, crucialmente, quanto tempo leva cada etapa. Antes de criar qualquer índice, execute a consulta abaixo e anote o tempo de execução. Este será o nosso **baseline** de comparação.

```
EXPLAIN ANALYZE
SELECT
  categoria,
  SUM(valor_total)
FROM fato_vendas
WHERE data_venda BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY categoria;
```

Sem índice, o PostgreSQL provavelmente utilizará um **Sequential Scan** (Seq Scan) — lendo todos os 1 milhão de registros linha por linha para encontrar os que se encaixam no filtro de data. Em tabelas grandes, esta é a operação mais lenta possível. Anote o campo `Execution Time` exibido ao final do plano de execução.

O que observar no EXPLAIN ANALYZE

- **Seq Scan:** Leitura sequencial — indica ausência de índice adequado
- **Index Scan:** Uso do índice — muito mais eficiente para filtros seletivos
- **Execution Time:** Tempo total de execução em milissegundos
- **Rows Removed by Filter:** Quantidade de registros descartados após leitura



Criando Índices e Medindo o Impacto

Após registrar o tempo sem índice, criaremos um índice na coluna `data_venda`. Um índice funciona como o índice de um livro: em vez de ler todas as páginas, você vai direto à página desejada. No PostgreSQL, índices do tipo **B-Tree** (padrão) são excelentes para filtros de igualdade e de intervalo como o nosso `BETWEEN`. A criação do índice é simples:

```
CREATE INDEX idx_data_venda ON fato_vendas(data_venda);
```

Após criar o índice, execute novamente o `EXPLAIN ANALYZE` com a mesma consulta. Desta vez, observe se o plano de execução mudou de `Seq Scan` para `Index Scan`. O ganho de performance pode ser dramático: consultas que levavam segundos podem passar a executar em dezenas de milissegundos.

✗ Sem Índice

Sequential Scan em 1M de registros. Tempo elevado. Todos os registros são lidos mesmo que apenas uma fração atenda ao filtro de data.

✓ Com Índice

Index Scan direto nos registros relevantes. Tempo drasticamente reduzido. O PostgreSQL acessa apenas as páginas com os dados do intervalo solicitado.



Desafio Extra (Opcional): Crie um índice composto em `(data_venda, categoria)` e compare novamente. Índices compostos podem ser ainda mais eficientes quando a consulta filtra e agrupa pelas mesmas colunas: `CREATE INDEX idx_data_categoria ON fato_vendas(data_venda, categoria);`

Discussão Final e Perguntas para Debate

Parabéns por concluir o exercício! Agora é hora de consolidar o aprendizado com uma discussão em grupo. As perguntas abaixo foram elaboradas para estimular o pensamento crítico sobre as decisões de design que você tomou ao longo da prática. Não existem respostas únicas — o objetivo é desenvolver o raciocínio analítico necessário para tomar boas decisões em projetos reais de BI.

1 Qual a vantagem do SCD Tipo 2?

Como o histórico de categorias preservado na `dim_produto` impacta a precisão de relatórios históricos? O que seria perdido se usássemos apenas um `UPDATE` simples?

2 Quando usar Materialized Views?

Existe um trade-off entre performance e atualidade dos dados. Em quais cenários a latência do `REFRESH` seria aceitável ou inaceitável?

3 Índices sempre melhoram tudo?

Índices consomem espaço em disco e tornam operações de `INSERT/UPDATE` mais lentas. Como decidir quais colunas indexar em um ambiente com alto volume de escrita?

4 Como isso afeta dashboards do Power BI?

De que forma Materialized Views pré-agregadas e índices bem planejados reduzem o tempo de carregamento de relatórios conectados diretamente ao PostgreSQL via DirectQuery?

"A diferença entre um banco de dados lento e um banco de dados rápido raramente está no hardware — está nas decisões de modelagem e indexação feitas pelo profissional de dados."